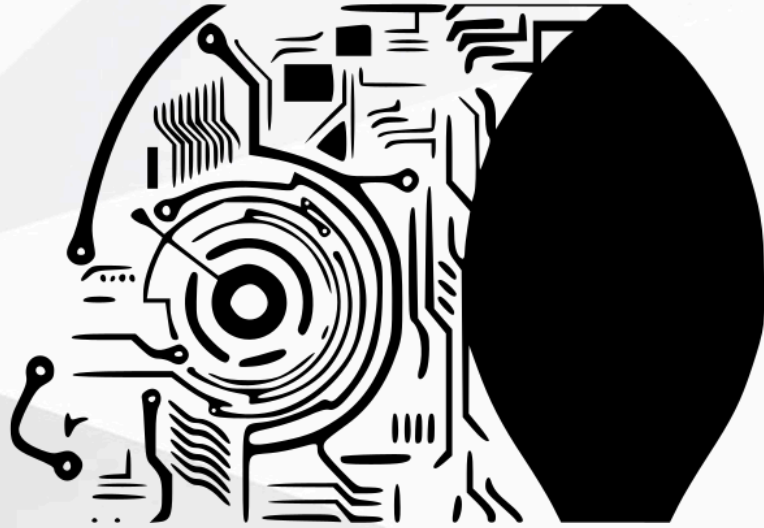




MODELOS DE I.A.

<https://martinezpenya.es/ModelosIA/>

© 2025 David Martínez licensed under CC BY-NC-SA 4.0

UD00: Presentación y curso rápido de Docker

Modelos de Inteligencia Artificial

version: 2026-10-01

¿Qué veremos?

1. El curso, el módulo y cómo se evalúa
2. El calendario del curso
3. Por qué los contenedores
4. Docker: imágenes, contenedores y volúmenes
5. Dockerfile y Compose
6. El entorno de prácticas del curso

El módulo 5071

Código	5071 · Modelos de Inteligencia Artificial
Duración	90 h · 3 h/semana · 4 ECTS
Curso	Especialización en IA y Big Data
Resultados de aprendizaje	6 propios (RA1-RA6) + el proyecto intermodular (RA7)

“ Del 1 de octubre al 28 de mayo. Clases de **lunes a jueves**: los viernes son para tareas y talleres. ”

Cómo se evalúa

- Cada **RA** se califica de **1 a 10, sin decimales**
 - Nota de cada RA = **40 %** tareas, talleres y ejercicios + **60 %** prueba escrita
 - **Una prueba por RA** en Moodle, al cerrar cada unidad
 - Hace falta **5 o más en CADA RA** para aprobar el módulo
- “ Puedes aprobar las dos evaluaciones y suspender el módulo por **un solo RA.** ”

Esta unidad no se califica... pero es obligatoria

Tres entregables, marcados **hecho / no hecho**:

1. **Taller 1** · verificación del entorno → memoria en PDF
 2. **Taller 2** · contenedor de prácticas → informe breve
 3. **Notebook** de la unidad → con las respuestas de la actividad
- “ Son la prueba de que tu entorno funciona. Sin ellos no se pueden hacer las prácticas de la UD02. ”

El calendario, en un vistazo

Cuándo	Qué
1 – 8 oct	UD00 · presentación y Docker
12 oct – 3 dic	UD01 y UD02
7 dic – 14 ene	UD05 · sistemas expertos
18 ene – 11 mar	UD03 (PLN) y UD04 (robótica)
15 mar – 23 abr	UD06 · ética y legalidad
26 abr – 28 may	Proyecto intermodular (RA7)

“Navidad: 22 dic – 6 ene · Pascua: 25 mar – 5 abr · Fallas: 17 y 18 de marzo

El problema: «en mi máquina funciona»

- Cada equipo tiene un Python distinto, con versiones distintas de cada biblioteca
 - Un notebook que va en clase **falla en casa**, y al revés
 - Reproducir un resultado ajeno se vuelve imposible
 - En IA esto es más grave: los resultados **no son comparables**
- “ La IA se hace con **entornos reproducibles**. ”

Contenedor frente a máquina virtual

	Máquina virtual	Contenedor
Qué virtualiza	Hardware completo	Solo el proceso
Sistema operativo	Uno propio por máquina	Comparte el kernel del anfitrión
Arranque	Minutos	Segundos
Tamaño	Gigabytes	Megabytes

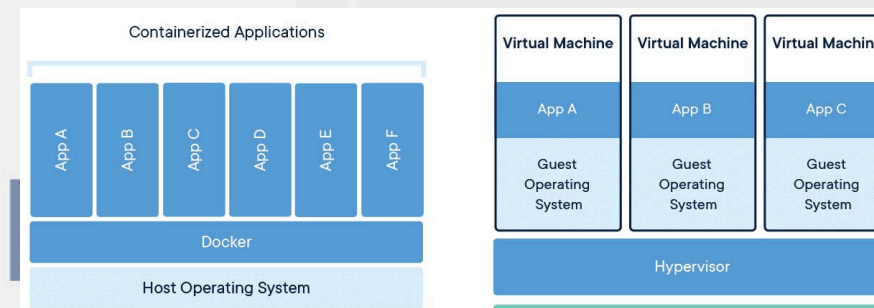
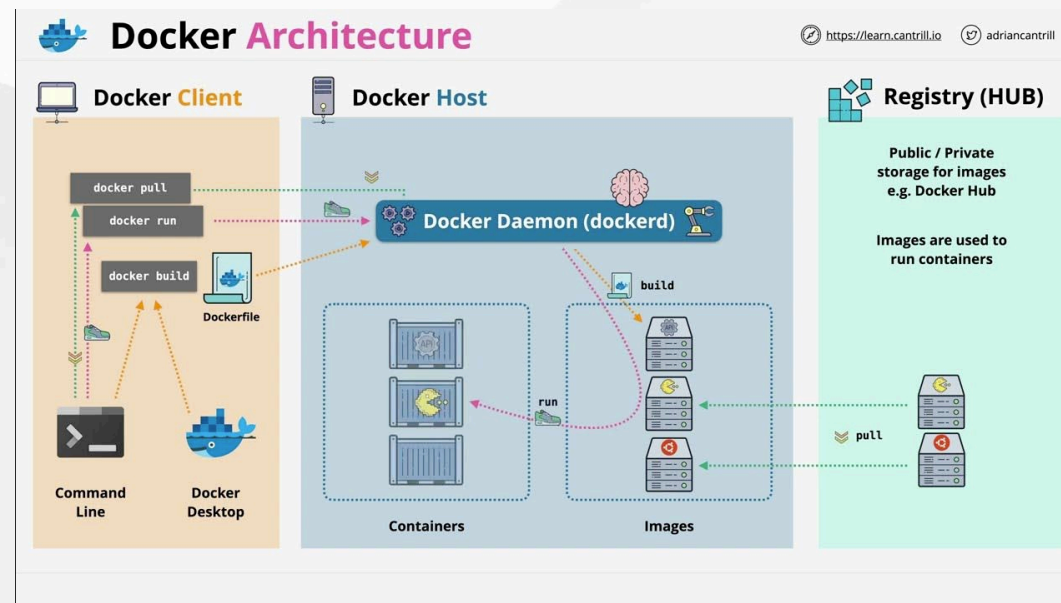


Imagen y contenedor

- **Imagen:** la plantilla, inmutable, hecha de **capas**
- **Contenedor:** una **instancia en ejecución** de esa imagen
- **Registro** (Docker Hub): donde viven las imágenes



Los comandos que usarás el 90 % del tiempo

```
docker run -it --name mi-python python:3.12 bash    # crear y entrar
docker ps -a                                         # qué contenedores tengo
docker images                                        # qué imágenes tengo
docker start / stop / rm                           # ciclo de vida
docker run -d -p 8080:80 nginx                      # servicio en segundo plano
```

“ **-it** interactivo • **-d** en segundo plano • **-p** publicar puerto •
--rm borrar al salir ”

Volúmenes: que los datos sobrevivan

Un contenedor es **desechable**: lo que escribes dentro desaparece al borrarlo.

```
docker run --rm -v "$PWD":/app -w /app python:3.12 python app.py
```

- **bind mount**: una carpeta tuya se ve dentro del contenedor
- **volumen nombrado**: Docker gestiona el almacenamiento

“ Tus notebooks viven **en tu equipo**, no dentro del contenedor. ”

Dockerfile: la receta

```
FROM python:3.12-slim
WORKDIR /home/mia
COPY requirements-ia.txt .
RUN pip install -r requirements-ia.txt
EXPOSE 8888
CMD ["jupyter", "notebook", "--ip=0.0.0.0"]
```

“ Cada instrucción es una **capa** y las capas se **cachean**: por eso las dependencias van antes que el código, para no reinstalarlas en cada cambio. ”

Compose: describir en vez de recordar

```
services:
  mia:
    build: .
    ports:
      - "8888:8888"
    volumes:
      - ../notebooks:/home/mia
```

```
docker compose up -d      # levantar
docker compose down       # parar y limpiar
```

El mismo código, dos entornos

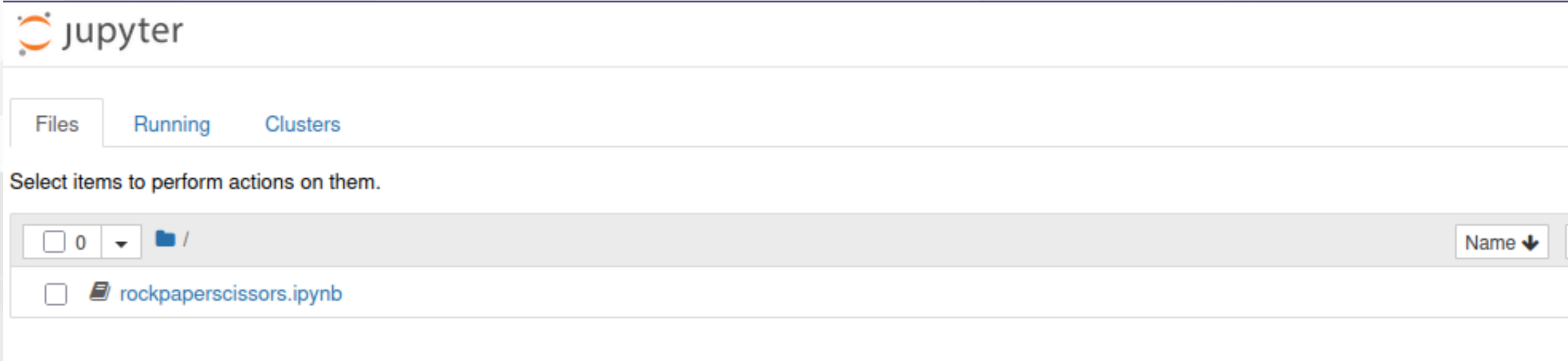
experta es el motor de reglas de la UD02 y la UD05. Última versión: **2019**.

```
docker run --rm python:3.9-slim bash -c "pip install -q experta && python -c 'import experta'"
docker run --rm python:3.12-slim bash -c "pip install -q experta && python -c 'import experta'"
```

- En 3.9 **funciona**; en 3.12 falla: **collections.Mapping** desapareció en Python 3.10
- Se arregla con **tres líneas** antes del import

“ Tres lecciones: el entorno importa, las dependencias se abandonan, y se puede convivir con ello.

Nuestro entorno de prácticas



- Un **Dockerfile** con **Python 3.12** y las bibliotecas del curso
- Un **docker-compose.yml** que publica **Jupyter** en el puerto 8888
- Un volumen para que **tus notebooks** queden en tu equipo

¿Y ahora?

1. Instala Docker y haz el **Taller 1**
2. Levanta el entorno con el **Taller 2**
3. Ejecuta el **notebook** de la unidad y entrégalo

“ En la **UD01** empezamos con los sistemas de IA: qué son, dónde se aplican y qué técnicas usan.